

Tensor Network Simulation of Grover’s Algorithm: Overcoming the Statevector Memory Wall

Veselin Petrov Nikolaev
University of Luxembourg
Faculty of Science, Technology and Medicine
Luxembourg, Luxembourg
veselin.nikolaev.001@student.uni.lu

Abstract

Previous work showed that classical statevector simulation of Grover’s algorithm reached a memory limit at approximately $n = 31$ – 32 qubits on GPU hardware. This project investigates whether tensor networks can provide a more memory-efficient alternative. The implementation uses the `quimb`, `cotengra`, and `kahypar` Python libraries instead of the NVIDIA `cuTensorNet` backend originally proposed.

The study measures three related properties of the tensor-network representation: the estimated memory required for exact contraction, the bond dimension needed for exact MPS convergence, and the difference between estimated and measured memory during real contractions.

The main findings are: (1) tensor networks did not provide a sustained memory advantage over statevector simulation for the tested range $n = 2$ – 11 ; (2) the minimum bond dimension for exact convergence increased from 4 to 32 between $n = 5$ and $n = 11$; and (3) measured memory was substantially larger than the simple width-based estimate, showing that theoretical contraction metrics do not directly predict practical memory usage.

Overall, the experiments suggest that Grover’s circuit is not a particularly favorable example for tensor-network compression at the sizes tested. The results are limited to the implementation and range studied, but they provide a useful example of how entanglement can reduce the practical advantage expected from tensor-network methods.

1 Introduction

Classical simulation of an n -qubit quantum state with a full statevector requires storing 2^n complex amplitudes. Previous work on Grover’s algorithm (BSP2) demonstrated this limitation experimentally, reaching approximately $n = 31$ – 32 qubits on GPU hardware before running out of memory. This motivates the question of whether a different representation of the same quantum state can reduce the practical cost.

Tensor networks provide one possible approach. Instead of storing all 2^n amplitudes explicitly, a large tensor can be factorized into a network of smaller tensors. The benefit depends on the structure of the state or circuit: when the required intermediate tensors or MPS bond dimensions remain small, the representation can be much more compact than a full statevector. `TensorNetwork.org` describes an MPS as a chain of smaller tensors connected by bond indices, with the bond dimension controlling the expressiveness of the representation [6].

Grover’s algorithm is useful for testing this idea because its circuit contains a structured oracle and diffusion operator, but its

operation also creates global interference across the search space. The goal of this project is therefore not to assume that tensor networks will work better, but to measure whether they actually provide an advantage for this particular algorithm.

1.1 Change of Backend

The original proposal specified NVIDIA `cuTensorNet` on GPU hardware. During implementation, the project instead used the CPU-based `quimb` + `cotengra` + `kahypar` stack. This was mainly a practical implementation decision: the chosen libraries made it possible to inspect contraction paths and perform the MPS experiments within the available development environment. The resulting comparison should therefore be understood primarily as a comparison of simulation methods, rather than a hardware-to-hardware benchmark.

The project uses three main libraries:

- (1) **quimb** (QUantum Information Many-Body): A Python library for tensor network operations, including quantum circuit evaluation via tensor contraction and via approximate matrix product state (MPS) simulation with a truncation parameter.
- (2) **cotengra** (Contraction Engine): A library that searches for good contraction orderings for tensor networks using randomized, hyperparameter-tuned heuristics (finding the truly optimal ordering is NP-hard in general).
- (3) **kahypar**: A hypergraph partitioner that `cotengra` can optionally use internally as one of its candidate methods for finding contraction orderings, alongside a simpler greedy method.

We address three research questions: (1) At what qubit count, if any, do tensor networks become more memory-efficient than statevector simulation? (2) How does the minimum bond dimension (a measure of entanglement in tensor decompositions) scale with qubit count? (3) At what qubit count does the tested tensor-network simulation become practically infeasible, and what observations help explain this behavior?

The rest of this report is organized as follows. Section 2 introduces Grover’s algorithm, tensor networks, and MPS bond dimension. Section 3 gives the relevant library and research context. Section 4 describes the implementation and experiments. Section 5 presents the results, and Section 6 discusses their interpretation. Section 7 highlights avenues for future work, and the report ends with the main conclusions in Section 8.

2 Theoretical Background

2.1 Grover’s Algorithm

We assume familiarity with Grover’s algorithm from prior work. Briefly: given a Boolean oracle marking exactly one target state $|x^*\rangle$ in a search space of $N = 2^n$ states, Grover’s algorithm uses repeated oracle-diffusion cycles to amplify the target’s amplitude. After $k_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor$ iterations, measurement yields $|x^*\rangle$ with high probability. The algorithm is implemented as n Hadamard gates (initialization), followed by k_{opt} iterations of (1) a phase oracle and (2) a diffusion operator.

2.2 Tensor Networks and Contraction

A quantum circuit can be represented as a tensor network: each quantum gate is a tensor (a multi-dimensional array), and evaluating the circuit means contracting the network (summing over shared indices).

Tensor definition. A tensor is a multi-dimensional array. A single-qubit gate is a 2×2 matrix (a rank-2 tensor); a two-qubit gate is $2 \times 2 \times 2 \times 2$ (rank-4).

Contraction. If tensor A has shape (d_a, d_b, d_c) and tensor B has shape (d_b, d_d) , their contraction along the shared index d_b produces a tensor C of shape (d_a, d_c, d_d) :

$$C_{a,c,d} = \sum_b A_{a,b,c} \cdot B_{b,d}$$

Contraction path. The order in which pairs of tensors are contracted determines both the memory cost (the size of intermediate tensors created during contraction) and the time cost (the number of scalar multiplications). Finding the truly optimal contraction path is NP-hard in general; cotengra instead runs a bounded, randomized search over candidate orderings and returns the best one found within a time/repeat budget. As discussed in Section 4, this search is stochastic and can return different-quality orderings across repeated runs on the same circuit, particularly for larger, more heavily decomposed circuits.

Contraction width (W). We define

$$W = \log_2 \left(\max_s |T_s| \right)$$

where $|T_s|$ is the size of the intermediate tensor created at contraction step s . W measures peak memory during contraction, expressed in qubits’ worth of dimension, directly comparable to the statevector’s $\log_2(2^n) = n$.

Contraction cost (C). C is $\log_{10}$ of the estimated total number of scalar multiplication operations across all contraction steps, for the chosen ordering.

2.3 Bond Dimension and Entanglement

An alternative approach to exact contraction is to approximate the state as a Matrix Product State (MPS), where the state is decomposed into a chain of small tensors connected by bonds, each with a bond dimension χ controlling both compression and accuracy.

Bond dimension definition. If the state is partitioned into left and right halves via singular value decomposition (SVD), the bond dimension is the number of singular values kept. Keeping all singular values gives an exact representation; truncating to the χ largest is a lossy compression. The minimum χ for which truncation

loses no information (to numerical precision) is what we measure and report.

Connection to entanglement. For a bipartition of a quantum state, the Schmidt rank is related to the MPS bond dimension. In particular, the entanglement entropy is bounded by $\log_2 \chi$. A product state has $\chi = 1$, while an arbitrary n -qubit state may require a bond dimension as large as $2^{\lfloor n/2 \rfloor}$ for an appropriate bipartition [6]. Therefore, increasing bond dimension provides a practical indication that the state is becoming harder to compress as an MPS.

2.4 Why Tensor Networks Can Fail for General Circuits

Two mechanisms can defeat tensor-network compression for a given circuit: (1) genuine entanglement growth, where the required bond dimension approaches $2^{n/2}$ regardless of contraction strategy, and (2) contraction path cost, where even a circuit with bounded entanglement can still be expensive to contract if a good ordering is hard to find or if gate decomposition inflates the network’s structure. Section 5 presents evidence for both mechanisms in this project’s data.

3 Related Work and Library Context

Tensor network simulation. Markov and Shi [2] established that contracting a tensor network computes quantum circuit amplitudes, and that circuits with bounded treewidth admit efficient (polynomial-time) contraction.

Contraction heuristics. Finding optimal contraction paths is NP-hard in general. cotengra [3] uses randomized, hyperparameter-tuned heuristics, optionally including hypergraph-partitioning-based methods via kahypar [4], to find good (not necessarily optimal) paths without exhaustive search.

Quantum simulation libraries. quimb [5] is a widely-used Python library for tensor-network quantum simulation, used here both for exact contraction (via its cotengra integration) and for approximate MPS simulation with explicit bond-dimension truncation.

Grover studies. Prior experimental work on Grover’s algorithm has focused on hardware demonstrations at small qubit counts or classical statevector simulation (as in BSP2). This project extends that line of work with a systematic tensor-network-based study.

4 Implementation and Methodology

4.1 Circuit Construction and Transpilation

Grover’s algorithm is constructed in Qiskit (reusing the oracle and diffuser construction from BSP2) and converted for use with quimb via the qiskit-quimb bridge. This bridge only recognizes elementary single- and two-qubit gates; Grover’s oracle and diffuser use an arbitrary-arity multi-controlled-X gate (mcx), which Qiskit’s simulator handles natively but the bridge does not. The circuit is therefore transpiled to a basis of rz, sx, x, and cx gates before conversion, which recursively decomposes each mcx into a chain of smaller gates. This decomposition is treated as a genuine part of the cost being measured, not a workaround to be discounted: it is the same gate-count blowup any concrete backend would face

when asked to execute an n -controlled gate using only elementary two-qubit interactions.

4.2 Search Stability of the Contraction Optimizer

cotengra’s randomized search (HyperOptimizer) can produce varying contraction width estimates across repeated runs on the same circuit, particularly for $n \geq 9$. To mitigate this variance, the search was run multiple independent times per qubit count, keeping the minimum width found (three repeats for $n \leq 9$; a single run for $n = 10, 11$ due to the search taking tens of minutes to hours). Repeated multi-trial search was used by default throughout, as single-shot greedy search proved unstable and occasionally returned implausibly poor orderings for larger circuits.

4.3 Bond Dimension Measurement via MPS Truncation

For each qubit count n and a set of randomly chosen target states, we run the Grover circuit through quimb’s MPS circuit simulator at a range of fixed maximum bond dimensions $\chi \in \{2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 48, 64\}$, and separately with χ uncapped as an exact reference. For each χ , we compare the resulting amplitude for the target bitstring against the uncapped reference amplitude, and record the smallest χ at which the error drops to (and stays at) zero to numerical precision. This is repeated for three independent random target states per n ; we report both the worst-case (maximum) and best-case (minimum) threshold across those three targets, since different target states can require different bond dimensions.

We note that this MPS-based approach is conceptually distinct from the exact-contraction approach used for W and C above: MPS truncation is lossy by construction (bond dimension is an approximation parameter), while the contraction-width approach is always exact. The two therefore answer different aspects of the same underlying research questions and are reported as separate, complementary results rather than one extending the other.

4.4 Real Memory Measurement

For small n (up to $n = 9$), we additionally perform the tensor network contraction for real (not just estimating its cost) and measure peak memory using Python’s tracemalloc module, which tracks memory allocated through Python’s memory allocator (this captures NumPy array allocations on standard builds, but can undercount memory allocated outside the Python allocator, such as some BLAS working buffers). To avoid wasting compute on contractions that would be infeasible to actually run, we only attempt real execution when the rehearsed width estimate implies a feasible memory footprint (below roughly 2 GB); the $n = 10, 11$ points in the main sweep, whose estimated widths implied far larger footprints, were excluded from this real-execution comparison for that reason.

4.5 Experimental Setup

To evaluate the tensor network representations systematically, experiments were structured as follows: For each qubit count n , the same Grover circuit was constructed, transpiled, and converted into a tensor network. The contraction optimizer was then used

to find a contraction path. From this path, the maximum intermediate tensor size (width W) and estimated contraction cost were recorded. Separately, the identical circuit was simulated as an MPS with different maximum bond dimensions.

The experiments measured scaling from $n = 2$ up to $n = 11$. Three random target states were tested for the bond-dimension experiments to capture variation, while other structural tests relied on a single representative target or randomized contraction paths. Real execution and memory tracking were conducted up to $n = 9$. Estimated paths were pushed to $n = 11$, beyond which the contraction optimizer itself became too computationally expensive to run reliably within the project’s constraints.

4.6 Experiment Suite

The project consists of four main experiments, summarized in Table 1:

- (1) **TN vs. statevector memory** ($n = 2$ –11): Compares theoretical statevector memory ($16 \cdot 2^n$ bytes) against the tensor network’s rehearsed memory estimate ($16 \cdot 2^W$ bytes). Answers research question 1. (Figure 7)
- (2) **Bond dimension growth** ($n = 5$ –11, three random targets per n): Tracks the minimum χ needed for exact MPS convergence, worst- and best-case. Answers research question 2. (Figure 8)
- (3) **Bond dimension convergence detail** (largest n tested, one target): Shows reconstruction error vs. χ for a single representative case, illustrating the mechanism behind Figure 8’s growth curve. (Figure 9)
- (4) **Rehearsed vs. real memory** ($n = 2$ –9): Compares cotengra’s width-based estimate against actually measured peak memory from real contraction. Informs research question 3 and validates the estimate itself. (Figure 10)

5 Results

5.1 Tensor Network vs. Statevector Memory (Figure 7)

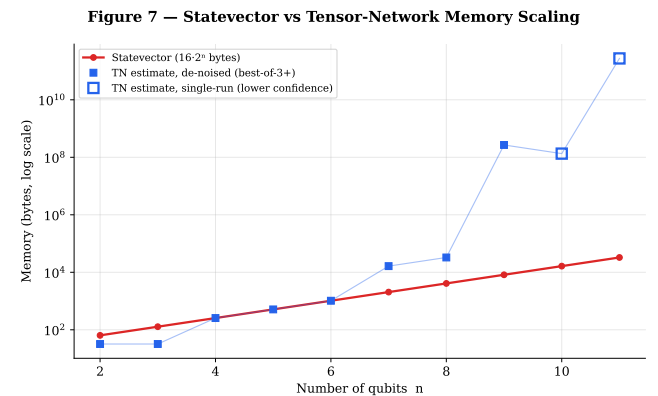


Figure 7: Statevector memory vs. tensor-network rehearsed memory estimate, $n = 2$ –11. Hollow markers at $n = 10, 11$ are single-run, lower-confidence estimates (Section 4.2).

Table 1: Summary of Experiments

Experiment	Qubit range	Repetitions	Main measurement
TN vs. statevector	2–11	Multiple optimizer runs	Estimated memory
Bond dimension	5–11	3 targets	Minimum exact χ
Convergence	11	1 target	Error vs. χ
Real memory	2–9	Multiple runs	Peak memory

Figure 7 compares statevector memory ($16 \cdot 2^n$ bytes) against the tensor network’s rehearsed memory estimate ($16 \cdot 2^W$ bytes) on a log scale for $n = 2$ –11. Solid, filled markers indicate values de-noised via multiple independent search repeats (see Section 4.2); hollow markers at $n = 10, 11$ indicate single-run estimates that should be read as order-of-magnitude only.

Key finding: Tensor networks do not achieve a sustained memory advantage. At low n (roughly $n \leq 6$), both approaches are relatively small, and the TN representation does not provide a clear advantage. Around $n = 7$, the estimated TN memory begins to increase rapidly. At $n = 10$ and 11, the optimizer itself becomes difficult to run, and the TN estimate exceeds the statevector figure by roughly six to seven orders of magnitude. Therefore, the expected theoretical advantage of tensor networks does not appear for this circuit. This directly answers research question 1: in the range tested, tensor networks do not become more memory-efficient than statevector simulation for Grover’s algorithm; if anything, the gap runs strongly in the opposite direction and widens with n .

5.2 Minimum Bond Dimension (Figure 8)

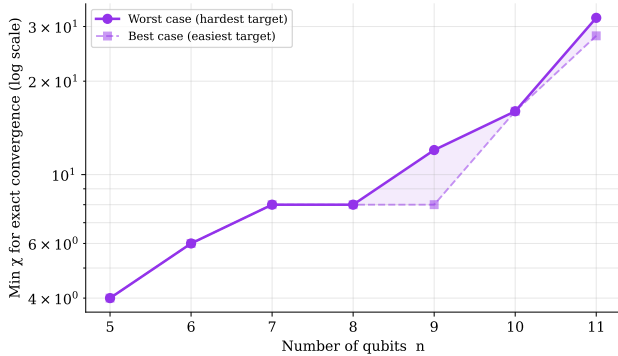
Figure 8 — Minimum Bond Dimension for Exact Convergence vs n 

Figure 8: Minimum bond dimension χ for exact MPS convergence vs. n , worst- and best-case across 3 random targets per n (shaded band shows the spread).

As detailed in Table 2 and illustrated in Figure 8, the experiments show the minimum bond dimension χ needed for exact MPS convergence, tested against three random target states per n for $n = 5$ –11; the shaded band shows the spread between best-case and worst-case targets.

Key finding: Bond dimension grows with n , with the clearest acceleration between $n = 10$ and $n = 11$ (16 to 32, a doubling). This

Table 2: Minimum bond dimension for exact convergence (worst case across 3 random targets)

n	$\chi_{\text{worst-case}}$
5	4
6	6
7	8
8	8
9	12
10	16
11	32

directly answers research question 2: the entanglement generated by Grover’s oracle and diffuser structure increases with qubit count, requiring a growing bond dimension to represent exactly.

5.3 Bond Dimension Convergence Detail (Figure 9)

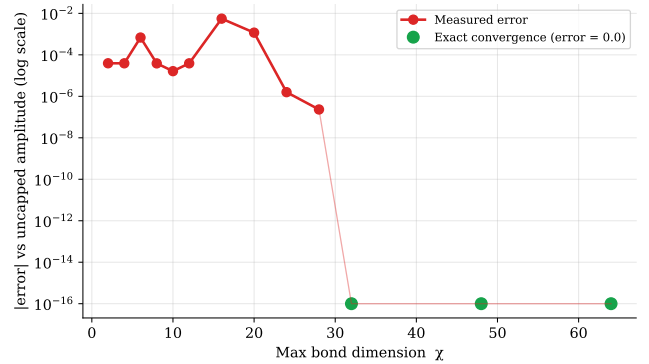
Figure 9 — Convergence vs Bond Dimension ($n=11$)

Figure 9: Reconstruction error vs. bond dimension χ for the largest n tested (one representative target). Green points converged exactly and are plotted at an arbitrary floor purely so the log scale renders.

Figure 9 shows reconstruction error (log scale) versus bond dimension χ for the largest n tested, for one representative target state. The error does not decay smoothly with increasing χ ; instead it remains large (and is not strictly monotonic — occasional non-monotonic bumps reflect that each gate’s MPS truncation is a locally, not globally, optimal decision) until a fairly sharp transition

region, after which it drops to numerical zero. This illustrates the mechanism behind Figure 8: convergence for this circuit behaves more like a threshold effect than a gradual improvement, so a coarse χ grid can misreport the true threshold if it only samples powers of two.

5.4 Rehearsed vs. Real Memory (Figure 10)

Figure 10 — Rehearsed Estimate vs Actually Measured Memory

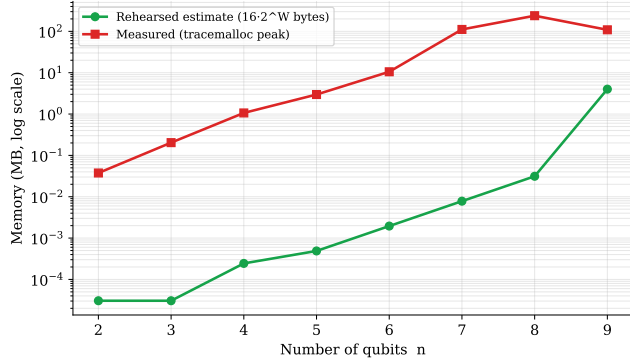


Figure 10: Rehearsed width-based memory estimate ($16 \cdot 2^W$ bytes) vs. actually measured peak memory (tracemalloc), $n = 2-9$.

Figure 10 compares cotengra’s width-based estimate ($16 \cdot 2^W$ bytes) against actually measured peak memory (via tracemalloc) from real contraction, for $n = 2-9$.

Key finding: real measured memory substantially exceeds the width-based estimate throughout this range — by roughly three orders of magnitude at small n , narrowing to roughly one to two orders of magnitude by $n = 9$. This is expected, not anomalous: W by definition captures only the size of the single largest intermediate tensor at any one contraction step, not the total memory footprint of the whole contraction process (which involves multiple coexisting tensors, plus NumPy/quimb/Python object overhead). This is an important methodological finding in its own right: the $16 \cdot 2^W$ formula should be read as a lower bound on required memory, not a tight prediction of actual usage.

6 Analysis and Discussion

6.1 Why Tensor Networks Struggle for Grover’s Algorithm

Two related factors, both visible in the data above, work against tensor-network compression for this circuit:

1. Increasing representation complexity. Figure 8 shows that the bond dimension required for exact MPS convergence increases with n . This is consistent with the idea that a larger bond dimension is needed when the state contains more correlations across the chosen bipartitions. The experiment does not by itself prove a specific entanglement mechanism, but it shows that the MPS representation becomes less compact as the circuit grows.

2. Gate-count increase from transpilation. As described in Section 4.1, the mcx decomposition required for tensor-network

compatibility increases the number of elementary gates. This makes the tensor network larger and can make contraction-path search more difficult, even before considering the bond dimension of the resulting state.

Together, these observations help explain why the tested tensor-network approach did not outperform statevector simulation for Grover’s algorithm. They should not be interpreted as a general limitation of tensor networks. Their effectiveness depends strongly on the structure of the circuit and on whether the resulting tensors remain sufficiently small.

6.2 Limitations

Several methodological limitations restrict the generalizability of these findings:

Hardware: The tensor-network experiments were CPU-based, while the BSP2 statevector result used GPU hardware. Thus, the results demonstrate that this specific CPU tensor-network implementation did not outperform the previous GPU statevector implementation, rather than definitively proving tensor networks are universally worse than statevectors on equivalent hardware.

Small qubit range: The tensor-network experiments only reached $n = 11$. This restricted scale does not allow for definitive claims about the asymptotic behavior of arbitrarily large Grover circuits.

Random target states: The bond-dimension experiment utilized three random target states per n . While reasonable for an initial empirical estimate, three samples are insufficient to fully characterize the behavior of every possible target state in the search space.

Contraction optimizer randomness: As noted in Section 4.2, the heuristic search for contraction paths introduces variance, meaning the recorded memory estimates reflect the best path found within the search budget rather than the guaranteed theoretical minimum.

Memory measurement: The use of tracemalloc captures memory allocated through Python’s allocator, but does not capture external memory such as certain low-level C or BLAS working buffers, rendering it a lower bound on the true memory footprint.

6.3 Practical Ceiling Reached in This Project

Real contraction was only feasible up to $n = 9$; single-search estimates were obtained up to $n = 11$ before search cost (tens of minutes to hours per contraction attempt) made going further impractical within this project’s scope. This ceiling is itself a data point: it is well short of BSP2’s statevector ceiling of $n = 31-32$ on GPU hardware, and directly reflects the entanglement-growth and search-instability findings above rather than being a separate, unrelated limitation.

7 Future Work

Several natural extensions to this project could provide a more comprehensive comparison of the representation methods:

GPU tensor-network simulation: Since the original proposal was based on NVIDIA cuTensorNet, a natural next step would be to repeat the experiments using cuTensorNet on GPU hardware and determine whether GPU acceleration changes the practical crossover point or extends the feasibility limit.

Different algorithms: Testing the same methodology on circuits that are more naturally suited to tensor networks (e.g., shallow circuits, low-entanglement circuits, or 1D quantum circuits) would help answer whether the scaling difficulties are inherent to tensor networks generally, or specifically tied to Grover’s circuit.

Better contraction optimization: Investigating whether a larger search budget or different hypergraph contraction heuristics produce significantly better contraction paths could be explored to see if it lowers the theoretical memory ceiling observed for $n \geq 10$.

Larger qubit counts: If better hardware or GPU acceleration were available, the experiments could be extended well beyond $n = 11$ to verify if the observed scaling trends hold true at larger sizes.

8 Conclusion

This project implemented and evaluated a tensor-network simulation of Grover’s algorithm using quimb, cotengra, and kahypar. The experiments measured estimated memory, bond dimension, and practical contraction behavior for $n = 2$ –11 qubits.

- (1) **At what qubit count do tensor networks become more memory-efficient?** None found in the tested range; by $n = 11$ tensor-network memory estimates exceed statevector memory by roughly six to seven orders of magnitude, with the gap widening as n increases.

- (2) **How does bond dimension scale?** It grows with n , from $\chi = 4$ at $n = 5$ to $\chi = 32$ at $n = 11$, most sharply between $n = 10$ and $n = 11$.

- (3) **Where and how does contraction become intractable?** Practically, real contraction was only feasible to $n = 9$ on the hardware used, and single-search estimates only to $n = 11$; the observations in Figure 9 demonstrate a sharp threshold-like convergence behavior in bond dimension, which contributes to the difficulty of scaling the simulation.

Overall, the results suggest that the tested tensor-network representation is not advantageous for Grover’s algorithm in this range. This should not be taken as a general failure of tensor networks. Rather, it illustrates an important limitation: their practical advantage depends on the circuit producing a representation with sufficiently small intermediate tensors or bond dimensions.

References

- [1] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [2] I. L. Markov and Y. Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [3] J. Gray. cotengra: Contraction Path Finding for Tensor Networks. <https://github.com/jcmgray/cotengra>, 2023.
- [4] S. Schlag et al. KaHyPar: A Multilevel Hypergraph Partitioner. *ACM Transactions on Mathematical Software*, 46(2):17, 2020.
- [5] J. Gray. quimb: A Python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.
- [6] Tensor Network. *Matrix Product State / Tensor Train*. TensorNetwork.org, accessed August 2026. <https://tensornetwork.org/mps/>.